

E-JAVA LAB ASSIGNMENT-3

20/03/2026

1. Write a Java program using ArrayList to store student details (USN, Name, Marks). Display all students and calculate the average marks.

```
import java.util.ArrayList;

// Student class
class Student {

    String usn;
    String name;
    double marks;

    // Constructor
    public Student(String usn, String name, double marks) {
        this.usn = usn;
        this.name = name;
        this.marks = marks;
    }

    // Method to display student details
    public void display() {
        System.out.println("USN : " + usn);
        System.out.println("Name : " + name);
        System.out.println("Marks : " + marks);
    }
}

// Main class
public class StudentArrayListDemo {
    public static void main(String[] args) {

        // Creating ArrayList
        ArrayList<Student> list = new ArrayList<>();

        // Adding students
        list.add(new Student("1BM21CS001", "Rahul", 85));
        list.add(new Student("1BM21CS002", "Anita", 90));
        list.add(new Student("1BM21CS003", "Kiran", 75));

        double total = 0;
```

```

// Display all students
for (Student s : list) {
    s.display();
    System.out.println();
    total += s.marks;
}

// Calculate average
double average = total / list.size();

System.out.println("Average Marks: " + average);
}
}

```

Output:

USN : 1BM21CS001

Name : Rahul

Marks : 85.0

USN : 1BM21CS002

Name : Anita

Marks : 90.0

USN : 1BM21CS003

Name : Kiran

Marks : 75.0

Average Marks: 83.33333333333333

2. Develop a program using ArrayList to maintain employee contact numbers and allow operations like add, remove, and search.

```
import java.util.ArrayList;
```

```
// Main class
```

```
public class EmployeeContacts {
    public static void main(String[] args) {
```

```
        // ArrayList to store contact numbers
```

```
        ArrayList<String> contacts = new ArrayList<>();
```

```
        // Adding contact numbers
```

```
        contacts.add("9876543210");
```

```
        contacts.add("9123456780");
```

```
        contacts.add("9988776655");
```

```

System.out.println("Contact List after adding:");
for (String num : contacts) {
    System.out.println(num);
}

// Removing a contact number
contacts.remove("9123456780");

System.out.println("\nContact List after removing:");
for (String num : contacts) {
    System.out.println(num);
}

// Searching for a contact number
String searchNumber = "9988776655";

if (contacts.contains(searchNumber)) {
    System.out.println("\nContact " + searchNumber + " found.");
} else {
    System.out.println("\nContact not found.");
}
}
}

```

Output:

Contact List after adding:

9876543210
9123456780
9988776655

Contact List after removing:

9876543210
9988776655

Contact 9988776655 found.

3. Create a program using LinkedList to store passenger names in a waiting list and allow insertion and deletion of passengers.

```
import java.util.LinkedList;
```

```
// Main class
```

```
public class PassengerWaitingList {
    public static void main(String[] args) {
```

```
        // Creating LinkedList
```

```
        LinkedList<String> waitingList = new LinkedList<>();
```

```
// Adding passengers
waitingList.add("Rahul");
waitingList.add("Anita");
waitingList.add("Kiran");

System.out.println("Waiting List after adding passengers:");
for (String name : waitingList) {
    System.out.println(name);
}

// Insert passenger at first position
waitingList.addFirst("Suresh");

System.out.println("\nAfter adding passenger at beginning:");
for (String name : waitingList) {
    System.out.println(name);
}

// Remove first passenger
waitingList.removeFirst();

System.out.println("\nAfter removing first passenger:");
for (String name : waitingList) {
    System.out.println(name);
}

// Remove specific passenger
waitingList.remove("Anita");

System.out.println("\nAfter removing Anita:");
for (String name : waitingList) {
    System.out.println(name);
}
}
```

Output:

Waiting List after adding passengers:

Rahul

Anita

Kiran

After adding passenger at beginning:

Suresh

Rahul

Anita

Kiran

After removing first passenger:

Rahul

Anita

Kiran

After removing Anita:

Rahul

Kiran

4. Implement a program using LinkedList to simulate a hospital patient queue where patients are added and removed in order.

```
import java.util.LinkedList;
```

```
// Main class
```

```
public class HospitalQueue {
```

```
    public static void main(String[] args) {
```

```
        // Creating LinkedList as Queue
```

```
        LinkedList<String> queue = new LinkedList<>();
```

```
        // Adding patients (enqueue)
```

```
        queue.add("Rahul");
```

```
        queue.add("Anita");
```

```
        queue.add("Kiran");
```

```
        System.out.println("Patients in Queue:");
```

```
        for (String patient : queue) {
```

```
            System.out.println(patient);
```

```
        }
```

```
        // Removing patient (dequeue - first patient)
```

```
        String servedPatient = queue.removeFirst();
```

```
        System.out.println("\nPatient Served: " + servedPatient);
```

```
        System.out.println("\nQueue after serving one patient:");
```

```
        for (String patient : queue) {
```

```
            System.out.println(patient);
```

```
        }
```

```
        // Adding another patient
```

```
        queue.add("Suresh");
```

```
        System.out.println("\nQueue after adding new patient:");
```

```
        for (String patient : queue) {
```

```
            System.out.println(patient);
```

```
        }
```

```
}  
}
```

Output:

Patients in Queue:

Rahul

Anita

Kiran

Patient Served: Rahul

Queue after serving one patient:

Anita

Kiran

Queue after adding new patient:

Anita

Kiran

Suresh

5. Write a Java program using HashSet to store email IDs of users and ensure duplicate registrations are not allowed.

```
import java.util.HashSet;
```

```
// Main class
```

```
public class EmailRegistration {  
    public static void main(String[] args) {
```

```
        // Creating HashSet
```

```
        HashSet<String> emails = new HashSet<>();
```

```
        // Adding email IDs
```

```
        emails.add("rahul@gmail.com");
```

```
        emails.add("anita@gmail.com");
```

```
        emails.add("kiran@gmail.com");
```

```
        emails.add("rahul@gmail.com"); // duplicate
```

```
        System.out.println("Registered Email IDs:");
```

```
        for (String email : emails) {
```

```
            System.out.println(email);
```

```
        }
```

```
        // Attempt to add duplicate email
```

```
        String newEmail = "anita@gmail.com";
```

```
        if (emails.add(newEmail)) {
```

```
            System.out.println("\nEmail registered successfully.");
```

```

    } else {
        System.out.println("\nDuplicate email! Registration not allowed.");
    }
}
}
}

```

Output:

Registered Email IDs:

rahul@gmail.com

anita@gmail.com

kiran@gmail.com

Duplicate email! Registration not allowed.

6. Develop a program to store course codes in a HashSet and display only unique course codes entered by the user.

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
// Main class
```

```
public class CourseCodes {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        // Creating HashSet
```

```
        HashSet<String> courses = new HashSet<>();
```

```
        System.out.print("Enter number of course codes: ");
```

```
        int n = sc.nextInt();
```

```
        sc.nextLine(); // consume newline
```

```
        // Taking input
```

```
        for (int i = 1; i <= n; i++) {
```

```
            System.out.print("Enter course code " + i + ": ");
```

```
            String code = sc.nextLine();
```

```
            courses.add(code); // duplicates automatically ignored
```

```
        }
```

```
        // Display unique course codes
```

```
        System.out.println("\nUnique Course Codes:");
```

```
        for (String c : courses) {
```

```
            System.out.println(c);
```

```
        }
```

```
        sc.close();
```

```
}  
}
```

Output:

Enter number of course codes: 5
Enter course code 1: CS101
Enter course code 2: CS102
Enter course code 3: CS101
Enter course code 4: CS103
Enter course code 5: CS102

Unique Course Codes:

CS101
CS102
CS103

7. Write a Java program using LinkedHashSet to store unique website visitor IP addresses while maintaining insertion order.

```
import java.util.LinkedHashSet;
```

```
// Main class
```

```
public class WebsiteVisitors {  
    public static void main(String[] args) {
```

```
        // Creating LinkedHashSet
```

```
        LinkedHashSet<String> ipAddresses = new LinkedHashSet<>();
```

```
        // Adding IP addresses
```

```
        ipAddresses.add("192.168.1.1");
```

```
        ipAddresses.add("192.168.1.2");
```

```
        ipAddresses.add("10.0.0.1");
```

```
        ipAddresses.add("192.168.1.1"); // duplicate
```

```
        ipAddresses.add("172.16.0.5");
```

```
        // Displaying unique IPs in insertion order
```

```
        System.out.println("Unique Visitor IP Addresses:");
```

```
        for (String ip : ipAddresses) {
```

```
            System.out.println(ip);
```

```
        }
```

```
    }
```

```
}
```

Output:

Unique Visitor IP Addresses:
192.168.1.1
192.168.1.2
10.0.0.1

8. Create a program using List interface and ArrayList to manage items in an online shopping cart (add, remove, display items).

```
import java.util.ArrayList;
import java.util.List;

// Main class
public class ShoppingCart {
    public static void main(String[] args) {

        // Using List interface with ArrayList
        List<String> cart = new ArrayList<>();

        // Adding items to cart
        cart.add("Laptop");
        cart.add("Headphones");
        cart.add("Mouse");

        System.out.println("Items in Cart:");
        for (String item : cart) {
            System.out.println(item);
        }

        // Removing an item
        cart.remove("Headphones");

        System.out.println("\nCart after removing Headphones:");
        for (String item : cart) {
            System.out.println(item);
        }

        // Adding another item
        cart.add("Keyboard");

        System.out.println("\nCart after adding Keyboard:");
        for (String item : cart) {
            System.out.println(item);
        }
    }
}
```

Output:

Items in Cart:

Laptop

Headphones

Mouse

Cart after removing Headphones:

Laptop

Mouse

Cart after adding Keyboard:

Laptop

Mouse

Keyboard

9. Write a Java program to create a thread that prints numbers from 1 to 10 with a delay of 1 second.

// Thread class

```
class NumberThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i <= 10; i++) {  
            System.out.println(i);
```

```
            try {
```

```
                Thread.sleep(1000); // 1000 ms = 1 second delay
```

```
            } catch (InterruptedException e) {
```

```
                System.out.println(e);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

// Main class

```
public class ThreadDemo {
```

```
    public static void main(String[] args) {
```

```
        NumberThread t1 = new NumberThread();
```

```
        t1.start(); // start thread
```

```
    }
```

```
}
```

Output:

1

2

3

4

5

6

7

8

9

10. Write a program to display the name and priority of the main thread and modify its priority.

```
// Main class
public class MainThreadDemo {
    public static void main(String[] args) {

        // Getting main thread reference
        Thread t = Thread.currentThread();

        // Display default name and priority
        System.out.println("Default Thread Name   : " + t.getName());
        System.out.println("Default Thread Priority : " + t.getPriority());

        // Modifying thread name and priority
        t.setName("MyMainThread");
        t.setPriority(7);

        // Display modified values
        System.out.println("\nAfter Modification:");
        System.out.println("Thread Name     : " + t.getName());
        System.out.println("Thread Priority : " + t.getPriority());
    }
}
```

Output:

```
Default Thread Name   : main
Default Thread Priority : 5
```

After Modification:

```
Thread Name     : MyMainThread
Thread Priority : 7
```

11. Develop a program where three threads simulate three machines printing documents simultaneously.

```
// Thread class
class Printer extends Thread {

    private String machineName;

    // Constructor
    public Printer(String machineName) {
        this.machineName = machineName;
    }

    // Run method
```

```

public void run() {

    for (int i = 1; i <= 5; i++) {
        System.out.println(machineName + " is printing document " + i);

        try {
            Thread.sleep(500); // delay for visibility
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

// Main class
public class PrintingSystem {
    public static void main(String[] args) {

        // Creating three threads (machines)
        Printer p1 = new Printer("Machine 1");
        Printer p2 = new Printer("Machine 2");
        Printer p3 = new Printer("Machine 3");

        // Starting threads
        p1.start();
        p2.start();
        p3.start();
    }
}

```

Output:

```

Machine 1 is printing document 1
Machine 2 is printing document 1
Machine 3 is printing document 1
Machine 1 is printing document 2
Machine 2 is printing document 2
Machine 3 is printing document 2
...

```

12. Write a program where two threads perform calculations and the main thread waits for them using join().

```

// Thread 1: Sum calculation
class SumThread extends Thread {

    public void run() {
        int sum = 0;

```

```

        for (int i = 1; i <= 5; i++) {
            sum += i;
        }

        System.out.println("Sum (1 to 5): " + sum);
    }
}

// Thread 2: Factorial calculation
class FactorialThread extends Thread {

    public void run() {
        int fact = 1;

        for (int i = 1; i <= 5; i++) {
            fact *= i;
        }

        System.out.println("Factorial of 5: " + fact);
    }
}

// Main class
public class JoinDemo {
    public static void main(String[] args) {

        SumThread t1 = new SumThread();
        FactorialThread t2 = new FactorialThread();

        t1.start();
        t2.start();

        try {
            // Main thread waits for both threads to finish
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Main thread finished execution.");
    }
}

```

Output:

Sum (1 to 5): 15

Factorial of 5: 120

Main thread finished execution.

13. Create a program demonstrating synchronization where multiple threads try to withdraw money from the same bank account.

```
// BankAccount class
class BankAccount {

    private double balance = 1000; // initial balance

    // Synchronized method
    public synchronized void withdraw(double amount) {

        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() +
                " is withdrawing ₹" + amount);

            try {
                Thread.sleep(500); // simulate processing time
            } catch (InterruptedException e) {
                System.out.println(e);
            }

            balance -= amount;
            System.out.println("Remaining Balance: ₹" + balance);
        } else {
            System.out.println(Thread.currentThread().getName() +
                " - Insufficient Balance!");
        }
    }
}

// Thread class
class Customer extends Thread {

    private BankAccount account;

    public Customer(BankAccount account, String name) {
        super(name);
        this.account = account;
    }

    public void run() {
        account.withdraw(700);
    }
}

// Main class
```

```

public class BankDemo {
    public static void main(String[] args) {

        BankAccount acc = new BankAccount();

        // Two threads accessing same account
        Customer c1 = new Customer(acc, "Customer 1");
        Customer c2 = new Customer(acc, "Customer 2");

        c1.start();
        c2.start();
    }
}

```

Output:

Customer 1 is withdrawing ₹700.0
 Remaining Balance: ₹300.0
 Customer 2 - Insufficient Balance!

14. Implement interthread communication using wait() and notify() to simulate a producer generating items and a consumer consuming them.

```

// Shared resource class
class Buffer {

    private int item;
    private boolean available = false;

    // Produce method
    public synchronized void produce(int value) {

        try {
            while (available) {
                wait(); // wait if item not yet consumed
            }

            item = value;
            System.out.println("Produced: " + item);

            available = true;
            notify(); // notify consumer

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

```

```

// Consume method
public synchronized void consume() {

    try {
        while (!available) {
            wait(); // wait if no item to consume
        }

        System.out.println("Consumed: " + item);

        available = false;
        notify(); // notify producer

    } catch (InterruptedException e) {
        System.out.println(e);
    }
}
}

```

```

// Producer thread
class Producer extends Thread {

    private Buffer buffer;

    public Producer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i);
        }
    }
}

```

```

// Consumer thread
class Consumer extends Thread {

    private Buffer buffer;

    public Consumer(Buffer buffer) {
        this.buffer = buffer;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
        }
    }
}

```

```

    }
}

// Main class
public class ProducerConsumerDemo {
    public static void main(String[] args) {

        Buffer buffer = new Buffer();

        Producer p = new Producer(buffer);
        Consumer c = new Consumer(buffer);

        p.start();
        c.start();
    }
}

```

Output:

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5

```

15. Write a Java program to divide two numbers and handle **ArithmeticException** if division by zero occurs.

```

import java.util.Scanner;

// Main class
public class DivisionExceptionDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter first number: ");
            int a = sc.nextInt();

            System.out.print("Enter second number: ");
            int b = sc.nextInt();

            int result = a / b; // may cause ArithmeticException
        }
    }
}

```

```

        System.out.println("Result: " + result);

    } catch (ArithmeticException e) {
        System.out.println("Error: Cannot divide by zero!");
    }

    sc.close();
}
}

```

Output 1:

Enter first number: 10
Enter second number: 2
Result: 5

Output 2:

Enter first number: 10
Enter second number: 0
Error: Cannot divide by zero!

16. Develop a program that reads an array element and handles both `ArrayIndexOutOfBoundsException` and `NumberFormatException`.

```

import java.util.Scanner;

// Main class
public class ArrayExceptionDemo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Array initialization
        int[] arr = {10, 20, 30, 40, 50};

        try {
            System.out.print("Enter index: ");
            int index = Integer.parseInt(sc.nextLine()); // may cause
            NumberFormatException

            System.out.println("Element: " + arr[index]); // may cause
            ArrayIndexOutOfBoundsException

        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid input! Please enter a number.");

        } catch (ArrayIndexOutOfBoundsException e) {

```

```

        System.out.println("Error: Index out of range!");
    } finally {
        System.out.println("Program executed.");
    }

    sc.close();
}
}

```

Output1:

Enter index: 2

Element: 30

Program executed.

Output 2:

Enter index: abc

Error: Invalid input! Please enter a number.

Program executed.

Output 3:

Enter index: 10

Error: Index out of range!

Program executed.

17. Create a custom exception class called **InsufficientBalanceException** and throw it if withdrawal amount exceeds account balance.

// Custom Exception class

```

class InsufficientBalanceException extends Exception {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}

```

// BankAccount class

```

class BankAccount {

    private double balance;

    // Constructor
    public BankAccount(double balance) {
        this.balance = balance;
    }
}

```

// Withdraw method

```

    public void withdraw(double amount) throws InsufficientBalanceException
    {
        if (amount > balance) {
            throw new InsufficientBalanceException("Withdrawal amount exceeds
balance!");
        } else {
            balance -= amount;
            System.out.println("Amount Withdrawn: ₹" + amount);
            System.out.println("Remaining Balance: ₹" + balance);
        }
    }
}

// Main class
public class CustomExceptionDemo {
    public static void main(String[] args) {

        BankAccount acc = new BankAccount(1000);

        try {
            acc.withdraw(1500); // exceeds balance
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

```

Output:
Exception: Withdrawal amount exceeds balance!

18. Write a program to read data from a file and ensure the file is closed using a finally block.

```

import java.io.FileReader;
import java.io.IOException;

// Main class
public class FileReadDemo {
    public static void main(String[] args) {

        FileReader fr = null;

        try {
            // Open file
            fr = new FileReader("sample.txt");

            int ch;

```

```

        System.out.println("File Contents:");

        // Read file character by character
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }

    } catch (IOException e) {
        System.out.println("Error: " + e.getMessage());
    }

    } finally {
        // Ensure file is closed
        try {
            if (fr != null) {
                fr.close();
                System.out.println("\nFile closed successfully.");
            }
        } catch (IOException e) {
            System.out.println("Error while closing file.");
        }
    }
}

```

Output:
 File Contents:
 Hello World
 Java File Handling

File closed successfully.

19. Write a program to write student details to a file and read them back using byte streams.

```

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

// Main class
public class StudentFileDemo {
    public static void main(String[] args) {

        String data = "USN: 1BM21CS001\nName: Rahul\nMarks: 85";

        // Writing to file using byte stream
        try {
            FileOutputStream fos = new FileOutputStream("student.txt");

```

```

        fos.write(data.getBytes()); // convert string to bytes
        fos.close();

        System.out.println("Data written to file successfully.");

    } catch (IOException e) {
        System.out.println("Error writing file.");
    }

    // Reading from file using byte stream
    try {
        FileInputStream fis = new FileInputStream("student.txt");

        int i;
        System.out.println("\nReading from file:");

        while ((i = fis.read()) != -1) {
            System.out.print((char) i);
        }

        fis.close();

    } catch (IOException e) {
        System.out.println("Error reading file.");
    }
}

```

Output:
Data written to file successfully.

Reading from file:
USN: 1BM21CS001
Name: Rahul
Marks: 85

20. Develop a Java program to serialize and deserialize an Employee object using ObjectOutputStream and ObjectInputStream.

```

import java.io.*;

// Employee class (must implement Serializable)
class Employee implements Serializable {

    int id;
    String name;

```

```

double salary;

// Constructor
public Employee(int id, String name, double salary) {
    this.id = id;
    this.name = name;
    this.salary = salary;
}

// Display method
public void display() {
    System.out.println("ID : " + id);
    System.out.println("Name : " + name);
    System.out.println("Salary : ₹" + salary);
}

// Main class
public class SerializationDemo {
    public static void main(String[] args) {

        Employee emp = new Employee(101, "Rahul", 50000);

        // Serialization
        try {
            ObjectOutputStream oos =
                new ObjectOutputStream(new FileOutputStream("employee.ser"));

            oos.writeObject(emp);
            oos.close();

            System.out.println("Object serialized successfully.");
        } catch (IOException e) {
            System.out.println("Serialization error.");
        }

        // Deserialization
        try {
            ObjectInputStream ois =
                new ObjectInputStream(new FileInputStream("employee.ser"));

            Employee e = (Employee) ois.readObject();
            ois.close();

            System.out.println("\nObject deserialized successfully.");
            e.display();
        }
    }
}

```

```
        } catch (IOException | ClassNotFoundException e) {  
            System.out.println("Deserialization error.");  
        }  
    }  
}
```

Output:

Object serialized successfully.

Object deserialized successfully.

ID : 101

Name : Rahul

Salary : ₹50000.0